



Universidad Alonso De Ojeda

Software Design

Autor: Blower Bosques C.I. 31.236.554
Carrera: Ingeniería en computación

Caracas, marzo 2026

1. Definición y Alcance del Software Design

El **Software Design** (Diseño de Software) es la etapa del ciclo de vida del desarrollo de software donde se transforman los requisitos funcionales y no funcionales en una representación técnica estructurada que sirve como blueprint para la implementación. No se trata meramente de "cómo escribir código", sino de establecer la arquitectura, la organización de componentes, las interfaces, las estructuras de datos y los algoritmos que conformarán el sistema.

Desde la perspectiva de la ingeniería de software, el diseño ocupa el espacio intermedio entre el **análisis de requisitos** (el "qué") y la **implementación** (el "cómo" en términos de código fuente). Es aquí donde se toman decisiones críticas que determinarán la calidad, mantenibilidad, escalabilidad y robustez del sistema final.

El diseño de software abarca dos niveles principales:

- **Diseño Arquitectónico (High-Level Design - HLD):** Define la estructura general del sistema, sus subsistemas principales, sus interfaces externas y las relaciones entre componentes de alto nivel.
- **Diseño Detallado (Low-Level Design - LLD):** Especifica la lógica interna de cada módulo, las estructuras de datos específicas, los algoritmos y las interfaces entre clases o funciones.

2. Importancia del Software Design

2.1. Fundamento de la Calidad del Software

La calidad del software no surge accidentalmente durante la codificación; es el resultado directo de decisiones de diseño conscientes. El diseño establece las bases para atributos de calidad fundamentales:

- **Mantenibilidad:** Un buen diseño modular permite localizar y modificar componentes sin afectar al resto del sistema (principio de localidad de cambios).
- **Escalabilidad:** La arquitectura debe permitir el crecimiento horizontal o vertical sin reescrituras masivas.
- **Confiabilidad:** El diseño incorpora mecanismos de manejo de errores, redundancia y recuperación ante fallos.
- **Usabilidad:** Aunque parezca exclusivo del frontend, el diseño de APIs y interfaces de servicio impacta directamente en la experiencia del desarrollador y, consecuentemente, del usuario final.
- **Eficiencia:** Decisiones sobre estructuras de datos y patrones de comunicación entre componentes determinan el consumo de recursos.

2.2. Mitigación de Complejidad

El software moderno enfrenta complejidad esencial (inherente al problema) y complejidad accidental (introducida por la solución). El diseño disciplinado utiliza técnicas de **abstracción**, **descomposición** y **encapsulamiento** para gestionar la complejidad, permitiendo que equipos de desarrollo trabajen en paralelo sobre componentes manejables sin perder la visión del sistema completo.

2.3. Reducción de Costos a Largo Plazo

Estudios empíricos (como los de Barry Boehm en "Software Engineering Economics") demuestran que el costo de corregir un defecto aumenta exponencialmente conforme avanza el ciclo de vida. Un error de diseño detectado en la fase de implementación puede costar 10x más que si se hubiera detectado durante el diseño; si llega a producción, el costo puede ser 100x superior. Invertir tiempo en diseño riguroso reduce drásticamente los costos totales de propiedad (TCO).

2.4. Comunicación y Coordinación

Los artefactos de diseño (diagramas UML, documentación de arquitectura, especificaciones de interfaces) sirven como medio de comunicación entre stakeholders técnicos y no técnicos, facilitando la alineación del equipo y la toma de decisiones informadas.

3. Características Fundamentales del Diseño de Software

3.1. Modularidad

La modularidad es la propiedad de un sistema de ser descompuesto en un conjunto de módulos cohesivos y débilmente acoplados. Un **módulo** es una unidad de programa que puede compilarse separadamente pero no ejecutarse independientemente (a diferencia de un componente).

Cohesión: Medida del grado en que los elementos de un módulo están relacionados funcionalmente. Desde la más baja hasta la más alta:

- Cohesión Coincidental (elementos no relacionados)
- Cohesión Lógica (elementos relacionados lógicamente, ej: manejo de entrada de diferentes tipos)
- Cohesión Temporal (elementos relacionados en tiempo)
- Cohesión Procedural (elementos que siguen una secuencia de ejecución)
- Cohesión de Comunicación (elementos que operan sobre la misma estructura de datos)
- Cohesión Secuencial (salida de uno es entrada del siguiente)
- Cohesión Funcional (todos los elementos contribuyen a una única función bien definida)

Acoplamiento: Medida de la interdependencia entre módulos. Tipos de acoplamiento (de más alto a más bajo):

- Acoplamiento de Contenido (un módulo modifica internamente otro)
- Acoplamiento Común (módulos comparten datos globales)
- Acoplamiento Externo (comparten una interfaz externa)
- Acoplamiento de Control (uno controla la lógica del otro)
- Acoplamiento de Marca (comparten estructuras de datos complejas)
- Acoplamiento de Datos (comparten solo datos primitivos)
- Sin acoplamiento (módulos totalmente independientes)

El objetivo es maximizar la cohesión y minimizar el acoplamiento.

3.2. Abstracción

La abstracción es el proceso de identificar las características esenciales de un objeto o sistema, ignorando los detalles irrelevantes. En diseño de software, se manifiesta en:

- **Abstracción de Procedimientos:** Definición de operaciones sin especificar su implementación (interfaces).
- **Abstracción de Datos:** Definición de tipos de datos y operaciones sobre ellos sin revelar su representación interna (tipos abstractos de datos).
- **Abstracción Arquitectónica:** Definición de subsistemas y sus responsabilidades sin detallar sus componentes internos.

3.3. Encapsulamiento (Information Hiding)

Acuñado por David Parnas, este principio establece que cada módulo debe ocultar sus decisiones de diseño a otros módulos, exponiendo únicamente una interfaz bien definida. Esto protege los detalles de implementación y permite modificarlos sin impactar al resto del sistema, siempre que la interfaz se mantenga estable.

3.4. Separación de Intereses (Separation of Concerns - SoC)

Atribuido a Edsger Dijkstra, este principio dicta que un sistema debe dividirse en partes distintas, cada una abordando un conjunto específico de información o interés. Ejemplos concretos:

- Separación entre lógica de negocio y lógica de presentación (MVC, MVVM).
- Separación entre persistencia de datos y procesamiento (Data Access Layer).
- Separación entre aspectos transversales como logging, seguridad o transacciones (Programación Orientada a Aspectos - AOP).

3.5. Portabilidad e Independencia

El diseño debe minimizar dependencias de plataformas específicas, hardware particular o tecnologías propietarias que puedan volverse obsoletas. Esto se logra mediante:

- Uso de estándares abiertos y protocolos bien establecidos.
- Implementación de capas de abstracción (abstraction layers) que aislen dependencias externas.
- Diseño basado en interfaces que permitan sustitución de implementaciones.

4. Principios Fundamentales del Diseño de Software

4.1. Principios SOLID

Estos cinco principios, formulados por Robert C. Martin (Uncle Bob), son fundamentales para el diseño orientado a objetos, pero sus conceptos son aplicables a paradigmas más amplios:

S - Single Responsibility Principle (SRP): "Un módulo debe tener una, y solo una, razón para cambiar." Esto significa que cada clase o módulo debe tener responsabilidad sobre una única parte de la funcionalidad del software, y esa responsabilidad debe estar completamente encapsulada. Violaciones comunes incluyen las "God Classes" que manejan múltiples responsabilidades no relacionadas.

O - Open/Closed Principle (OCP): "Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación." Esto se logra mediante el uso de abstracciones (interfaces o clases abstractas) que permiten añadir nuevos comportamientos sin alterar código existente y probado. Técnicas: herencia, composición, patrones Strategy o Template Method.

L - Liskov Substitution Principle (LSP): "Los objetos de una clase derivada deben poder sustituir a objetos de la clase base sin alterar la corrección del programa." Formulado por Barbara Liskov, este principio garantiza que la herencia se use correctamente. Violaciones ocurren cuando una subclase restringe el comportamiento de la superclase (ej: un Cuadrado que hereda de Rectángulo pero viola las invariantes de proporción).

I - Interface Segregation Principle (ISP): "Ningún cliente debe verse forzado a depender de métodos que no utiliza." Es preferible tener muchas interfaces específicas que una única interfaz general. Esto evita que los clientes dependan de funcionalidad irrelevante y reduce el impacto de cambios en la interfaz.

D - Dependency Inversion Principle (DIP): "Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones. Las abstracciones no deben depender de los detalles; los detalles deben depender de las abstracciones." Este principio es la base de la inyección de dependencias y la arquitectura hexagonal (ports and adapters).

4.2. Principio DRY (Don't Repeat Yourself)

Cada pieza de conocimiento debe tener una representación única, unificada y autoritativa en el sistema. La duplicación de código o lógica introduce inconsistencias y multiplica los puntos de mantenimiento. Sin embargo, hay que distinguir entre duplicación real (violación de DRY) y duplicación accidental (coincidencia superficial que puede variar independientemente).

4.3. Principio KISS (Keep It Simple, Stupid)

La simplicidad debe ser un objetivo clave en el diseño, y la complejidad innecesaria debe ser evitada. Esto no significa evitar soluciones sofisticadas cuando son necesarias, sino rechazar la complejidad que no aporta valor. Cada componente debe ser tan simple como sea posible, pero no más simple (paráfrasis de Einstein).

4.4. Principio de Mínimo Sorpresa (Principle of Least Astonishment)

Un componente debe comportarse de manera consistente con las expectativas del usuario o desarrollador. Las interfaces deben ser intuitivas, los nombres descriptivos, y los comportamientos predecibles. Esto reduce la curva de aprendizaje y los errores de uso.

4.5. Principio de Composición sobre Herencia

Favorecer la composición de objetos sobre la herencia de clases. La herencia crea acoplamiento fuerte y jerarquías rígidas difíciles de modificar. La composición permite mayor flexibilidad, permitiendo cambiar comportamientos en tiempo de ejecución y facilitando el testing unitario mediante mocks.

4.6. Ley de Demeter (Principle of Least Knowledge)

"Un objeto debe tener conocimiento limitado sobre otros objetos, interactuando solo con sus amigos inmediatos." Formalmente: un método de un objeto puede llamar métodos de:

1. El objeto mismo
2. Los parámetros del método
3. Cualquier objeto creado/instantiado dentro del método
4. Los atributos directos del objeto
5. Variables globales accesibles desde el objeto

Esto reduce el acoplamiento y evita cadenas de llamadas frágiles (train wreck: `objeto.getA().getB().getC().hacerAlgo()`).

5. Perspectivas y Enfoques del Diseño de Software

5.1. Perspectiva Estructural (Static View)

Se centra en la organización estática del sistema: clases, módulos, paquetes, componentes y sus relaciones. Utiliza diagramas de clases UML, diagramas de componentes y diagramas de paquetes. Aspectos clave:

- Jerarquías de herencia y composición.
- Visibilidad de miembros (público, protegido, privado).
- Dependencias entre módulos.
- Métricas de diseño: acoplamiento aferente (Ca) y eferente (Ce), inestabilidad, distancia de la secuencia principal.

5.2. Perspectiva Comportamental (Dynamic View)

Enfocada en cómo el sistema se comporta en tiempo de ejecución: interacciones entre objetos, flujos de control, estados y actividades. Utiliza:

- **Diagramas de Secuencia:** Interacciones temporales entre objetos.

- **Diagramas de Comunicación (Colaboración):** Enfocados en las relaciones estructurales durante la interacción.
- **Diagramas de Estado:** Transiciones de estados de objetos complejos.
- **Diagramas de Actividad:** Flujos de trabajo y procesos de negocio.

5.3. Perspectiva Arquitectónica

Visión de alto nivel que define la estructura fundamental del sistema, sus patrones arquitectónicos y estilos:

- **Arquitectura en Capas (Layered):** Separación en capas horizontales (presentación, lógica de negocio, persistencia).
- **Arquitectura Cliente-Servidor:** Distribución entre consumidores y proveedores de servicios.
- **Arquitectura Microservicios:** Descomposición en servicios pequeños, autónomos y desplegados independientemente.
- **Arquitectura Orientada a Eventos:** Comunicación asíncrona basada en eventos (Event-Driven Architecture - EDA).
- **Arquitectura Hexagonal (Ports and Adapters):** Aislamiento del dominio de la infraestructura.
- **Clean Architecture / Onion Architecture:** Dependencias que apuntan siempre hacia el dominio central.

5.4. Perspectiva de Datos

Centrada en la persistencia, estructuración y flujo de datos:

- Modelado de datos (ER, diagramas de clase de dominio).
- Estrategias de persistencia (relacional, NoSQL, cache).

- Patrones de acceso a datos (Repository, Unit of Work, Data Mapper).
- Consistencia, transaccionalidad y estrategias de concurrencia (optimista vs pesimista).

5.5. Perspectiva de Calidad (Quality Attributes)

No funcional, enfocada en atributos de calidad específicos:

- **Seguridad:** Autenticación, autorización, cifrado, auditoría.
 - **Rendimiento:** Latencia, throughput, escalabilidad.
 - **Disponibilidad:** Tolerancia a fallos, recuperación ante desastres.
 - **Mantenibilidad:** Modularidad, reusabilidad, analizabilidad.
 - **Usabilidad:** Accesibilidad, internacionalización, feedback al usuario.
-

6. Aplicaciones y Utilización del Software Design

6.1. Contextos de Aplicación

Desarrollo de Sistemas Empresariales (Enterprise Software): El diseño es crítico en sistemas ERP, CRM y plataformas bancarias donde la longevidad, integración con sistemas legados y cumplimiento normativo son esenciales. Se utilizan arquitecturas basadas en dominio (DDD - Domain-Driven Design) para modelar complejidades de negocio.

Sistemas de Tiempo Real y Embebidos: Diseño que considera restricciones temporales estrictas, determinismo y recursos limitados. Uso de patrones como Observer para manejo de eventos, y arquitecturas de scheduling (rate-monotonic, earliest deadline first).

Sistemas Distribuidos y Cloud-Native: Diseño para fallas inevitables (design for failure), consistencia eventual, particionamiento de redes (teorema CAP). Patrones como Circuit Breaker, Bulkhead, Saga para transacciones distribuidas, y CQRS (Command Query Responsibility Segregation).

Desarrollo de APIs y Microservicios: Diseño de contratos de API (REST, GraphQL, gRPC), versionado, estrategias de despliegue (blue-green, canary), y comunicación asíncrona mediante message brokers.

Sistemas de Inteligencia Artificial y Machine Learning: Diseño de pipelines de datos, arquitecturas de feature stores, model serving, y MLOps. Separación entre experimentación y producción, gestión de modelos y reproducibilidad.

6.2. Técnicas y Herramientas de Diseño

Lenguaje de Modelado Unificado (UML): Notación estándar para visualizar, especificar, construir y documentar artefactos de software. Diagramas esenciales:

- Clases (estructura estática)
- Secuencia (interacción temporal)
- Componentes (organización física)
- Despliegue (arquitectura física)
- Estados (comportamiento reactivo)
- Casos de uso (requisitos funcionales)

Domain-Driven Design (DDD): Enfoque de diseño centrado en el dominio de negocio, utilizando:

- **Lenguaje Ubicuo (Ubiquitous Language):** Vocabulario compartido entre desarrolladores y expertos de dominio.

- **Entidades y Value Objects:** Objetos con identidad vs objetos definidos por sus atributos.
- **Agregados:** Clusters de objetos tratados como unidad de consistencia.
- **Repositorios:** Abstracción de persistencia para agregados.
- **Servicios de Dominio:** Operaciones que no pertenecen naturalmente a entidades o value objects.
- **Eventos de Dominio:** Capturan ocurrencias significativas en el dominio.

Patrones de Diseño (GoF - Gang of Four): Soluciones reutilizables a problemas comunes:

- **Creacionales:** Singleton, Factory Method, Abstract Factory, Builder, Prototype.
- **Estructurales:** Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
- **Comportamiento:** Observer, Strategy, Command, Iterator, State, Template Method, Visitor, Mediator, Memento, Chain of Responsibility, Interpreter.

Arquitectura de Software: Patrones a nivel de sistema:

- MVC (Model-View-Controller)
- MVP (Model-View-Presenter)
- MVVM (Model-View-ViewModel)
- N-capas (N-tier)
- Peer-to-Peer
- Space-Based Architecture
- Event Sourcing

- CQRS

6.3. Procesos de Diseño

Diseño Basado en Pruebas (Test-Driven Development - TDD): Ciclo Red-Green-Refactor donde el diseño emerge de la necesidad de hacer código testeable, promoviendo desacoplamiento y alta cohesión.

Behavior-Driven Development (BDD): Extensión de TDD que utiliza lenguaje natural estructurado (Given-When-Then) para definir comportamientos esperados, alineando diseño con requisitos de negocio.

Design Thinking aplicado a Software: Enfoque humano-centrado que involucra empatizar, definir, idear, prototipar y testear soluciones, particularmente útil para el diseño de UX/UI y arquitecturas que impactan directamente en la experiencia del usuario.

7. Métricas y Evaluación del Diseño

7.1. Métricas Orientadas a Objetos

- **Profundidad de la Jerarquía de Herencia (DIT):** Niveles de herencia; valores altos indican complejidad.
- **Número de Hijos (NOC):** Subclases directas; alto NOC sugiere reuso pero también fragilidad.
- **Acoplamiento entre Clases (CBO):** Número de clases acopladas a una clase dada.
- **Falta de Cohesión en Métodos (LCOM):** Medida de cuán relacionados están los métodos de una clase entre sí.

- **Respuesta para una Clase (RFC):** Número de métodos que potencialmente pueden ejecutarse en respuesta a un mensaje recibido.

7.2. Métricas de Complejidad

- **Complejidad Ciclomática (McCabe):** Número de caminos independientes a través del código, indicando complejidad de testing.
- **Complejidad Cognitiva:** Extensión que considera estructuras anidadas y flujos de control no lineales.

7.3. Análisis Estático y Revisiones de Diseño

- **Análisis de Dependencias:** Herramientas como NDepend, JDepend, Structure101 para visualizar y detectar ciclos de dependencias.
 - **Revisiones de Arquitectura (Architecture Reviews):** Evaluaciones formales de la arquitectura contra atributos de calidad.
 - **ATAM (Architecture Tradeoff Analysis Method):** Método sistemático para evaluar arquitecturas considerando múltiples atributos de calidad y sus trade-offs.
-

8. Desafíos Contemporáneos en el Diseño de Software

8.1. Escalabilidad Masiva (Web Scale)

Diseño para millones de usuarios concurrentes requiere:

- Sharding de bases de datos.
- Caché distribuida (Redis, Memcached).
- Colas de mensajes para desacoplamiento temporal.

- Arquitecturas serverless y auto-escalables.

8.2. Sistemas Concurrentes y Paralelos

Gestión de:

- Condiciones de carrera (race conditions).
- Deadlocks y livelocks.
- Starvation.
- Consistencia de caché en sistemas distribuidos. Uso de modelos como Actores (Akka, Orleans), Software Transactional Memory (STM), y programación reactiva (Reactive Streams).

8.3. Diseño para Evolución y Legacy

Estrategias para modernizar sistemas legacy:

- Patrón Strangler Fig (estrangular gradualmente el sistema antiguo).
- Anti-Corruption Layer (aislar dominio nuevo de legacy).
- Feature Toggles para despliegues incrementales.

8.4. Sostenibilidad y Green Software

Diseño consciente del consumo energético:

- Eficiencia algorítmica.
- Arquitecturas que reduzcan transferencia de datos innecesaria.
- Optimización de consultas y uso de recursos computacionales.

Conclusión

El Software Design es la disciplina que transforma la ingeniería de software de un artefacto artesanal en una ingeniería disciplinada. Su importancia radica en que las decisiones de diseño tienen efectos duraderos que perdurarán durante toda la vida útil del software, mucho más allá de la fase inicial de desarrollo.

Las características de modularidad, abstracción, encapsulamiento y separación de intereses proporcionan los mecanismos para gestionar la complejidad inherente al software moderno. Los principios SOLID, DRY, KISS y otros marcos conceptuales ofrecen heurísticas probadas para tomar decisiones de diseño robustas.

Las múltiples perspectivas —estructural, comportamental, arquitectónica, de datos y de calidad— exigen que el diseñador de software sea capaz de razonar sobre el sistema desde diferentes ángulos, utilizando notaciones formales como UML y técnicas avanzadas como Domain-Driven Design.

Finalmente, la aplicación práctica del diseño de software abarca desde sistemas empresariales tradicionales hasta arquitecturas cloud-native y sistemas de IA, requiriendo constante adaptación a nuevas tecnologías, paradigmas y restricciones, pero siempre fundamentado en principios atemporales de buena ingeniería.